For the latter, a hash table would be faster, but it would also be vulnerable to worst-case-input attacks, where an attacker could send packets from IPs hashing to the value: this would degenerate the hash table into an unsorted list (if concatenation[9] is used), transforming the hash table in a bottle neck and possibly slowing down packet resolution or the whole kernel.

3.5 Performance Analysis and Profiling

As we have seen in section 3.3, all API methods on randomized treaps require time proportional to the height of tree; as we know (see the introduction to this chapter and appendix C) in the worst case the height of a binary tree can be linear in the number of elements and, in fact, one of the problems with BSTs is that there are specific sequences of insertions that will certainly cause a tree to be skewed: this issue makes binary search trees particularly vulnerable to attacks, when used as dictionaries, because all an attacker needs to degrade the data structure's performance is to send an ordered sequence, which will cause the tree to degenerate into a linked list, having a single path from the root wo a leaf containing all the elements.

Randomized treaps offer a two-folded improvement: first and foremost, introducing randomness in the assignment of priorities prevents[10] attackers from being able to exploit known sequences.

But also, as we promised in section 3.4, it will give us on average a more balanced tree than plain binary search trees.